# Client Cybersecurity TPRA — Repository Guide

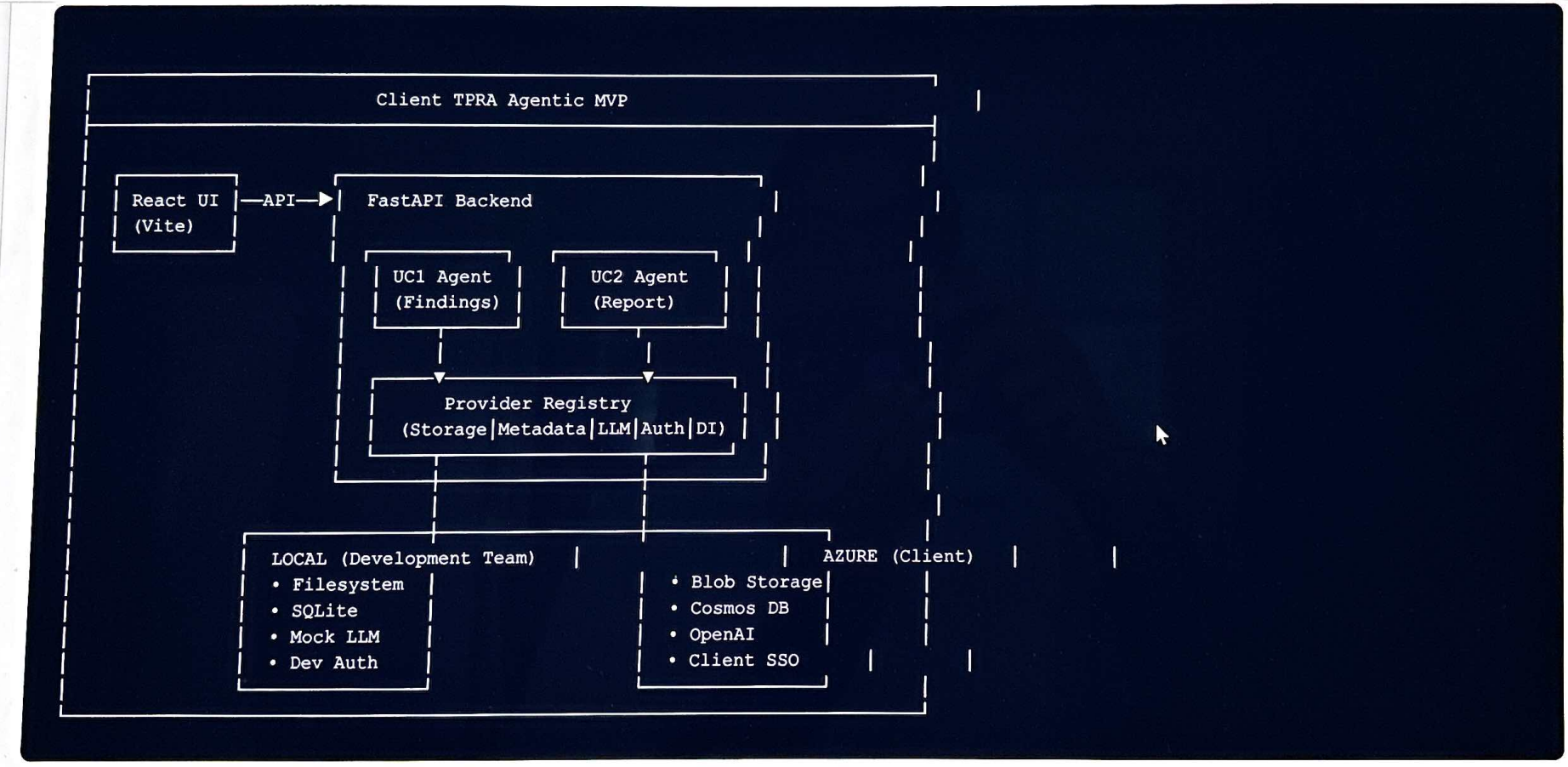
Third Party Risk Assessment Agentic MVP • Built by Development Team for Client

## What Is This Project?

This is a **human-in-the-loop agentic platform** for the client's Third Party Risk Assessment (TPRA) process. It has two AI agents:

- **UC1 — Structured Findings Package Agent:** Ingests raw vendor security findings (Excel/CSV/JSON/DOCX), normalizes them to a canonical schema, validates required fields, flags exceptions, and produces a structured output package.
- **UC2 — Draft TPRA Report Generation Agent:** Takes the approved findings from UC1, maps them to a report template, invokes an AI model to draft narrative sections, and produces a Word document draft for human review.

The architecture follows a **"build once, run anywhere"** pattern — provider interfaces abstract away infrastructure so the same codebase runs locally (mock LLM, SQLite, local filesystem) or on Azure (OpenAI, Cosmos DB, Blob Storage).


Client TPRA Agentic MVP
React UI
(Vite)
API
FastAPI Backend
UC1 Agent
(Findings)
UC2 Agent
(Report)
Provider Registry
(Storage|Metadata|LLM|Auth|DI)
LOCAL (Development Team)
• Filesystem
• SQLite
• Mock LLM
• Dev Auth
AZURE (Client)
• Blob Storage
• Cosmos DB
• OpenAI
• Client SSO

Backend (Python/FastAPI) Frontend (React/TypeScript) Infrastructure (Azure/Docker) Configuration Documentation Scripts & Automation

## Root-Level Files Config

| FILE | WHY IT EXISTS |
|---|---|
| `.gitignore` | Tells Git which files to never commit — Python bytecode, virtual envs, node_modules, build artifacts, secrets, runtime data. |
| `.env.example` | Template showing every environment variable the system supports. Copy to `.env` and fill in real values to configure locally. |
| `.env.local.example` | Pre-filled environment preset for Development Team local development (mock LLM, SQLite, local storage, dev auth). Copy as `.env` to start immediately. |
| `.env.client.example` | Pre-filled environment preset for the client's DaVinci cloud (Azure OpenAI, Cosmos DB, Blob Storage, Client SSO). Used during deployment. |
| `docker-compose.yml` | Defines two Docker services (`api` + `ui`) for one-command local full-stack startup. No Azure needed — everything runs with local/mock providers. |
| `README.md` | Primary project documentation: architecture overview, quickstart guide, API surface, repository layout, and deployment instructions. |

## Backend Python / FastAPI

### Build & Config Files

| FILE | WHY IT EXISTS |
| --- | --- |
| `backend/config.yaml` | Primary human-editable configuration: provider selection (storage, metadata, LLM, auth, doc-intel), Azure endpoints, local paths, CORS, observability settings. This is the single knob that switches between local-development and client-cloud. |
| `backend/config.client.example.yaml` | Example config preset for Client DaVinci. Copy over `config.yaml` when migrating to the client environment. |
| `backend/requirements.txt` | Full Python dependency list including Azure SDKs (Blob, Cosmos, Document Intelligence, AI Foundry). Used in Docker builds and CI. |
| `backend/requirements-dev.txt` | Minimal dependencies for local development and CI — excludes heavy Azure SDKs so `pip install` is fast when using mock providers. |
| `backend/Dockerfile` | Builds the production API container image (Python 3.11-slim, installs deps, copies app code, runs uvicorn). |
| `backend/.dockerignore` | Excludes .venv, .data, tests, and docs from the Docker build context to keep images small and secure. |
| `backend/pytest.ini` | Pytest configuration: enables async test mode, sets test discovery paths, configures markers. |
| `backend/demo_flow.py` | End-to-end demo script: generates sample data, exercises the full UC1→UC2 pipeline via the API, and downloads all outputs. Used for demos and smoke testing. |

### app/core/ — Foundation Layer

| FILE | WHY IT EXISTS |
| --- | --- |
| `app/core/config.py` | Loads configuration with a precedence chain: environment variables > config.yaml > .env file > defaults. Exposes typed settings objects to the rest of the app. |

| | |
|---|---|
| `app/core/exceptions.py` | Defines custom exception classes (TPRAError, ValidationError, AuthzError) so the API can return structured error responses. |
| `app/core/logging.py` | Configures structured JSON logging with correlation IDs for request tracing across services. |

### app/domain/ — Business Logic Models

| FILE | WHY IT EXISTS |
|---|---|
| `app/domain/models.py` | Canonical data models (Finding, Exception, Workspace, Run, AuditEvent) — the "nouns" of the system that every layer speaks in. |
| `app/domain/enums.py` | Enumerations for statuses (RunStatus, FindingStatus, ApprovalLevel), roles, and provider types — keeps magic strings out of code. |
| `app/domain/validation_rules.py` | Business rules for what makes a finding "valid" (required fields, allowed values, cross-field constraints). Used by UC1's validator step. |

### app/schemas/ — API Contracts

| FILE | WHY IT EXISTS |
|---|---|
| `app/schemas/api.py` | Pydantic models for API request/response shapes — ensures strict validation at the HTTP boundary and auto-generates OpenAPI docs. |

### app/api/ — HTTP Route Handlers

| FILE | WHY IT EXISTS |
|---|---|
| `app/api/deps.py` | FastAPI dependency injection — provides authenticated user context, provider instances, and services to route handlers. |

| | |
|---|---|
| app/api/v1/health.py | Health check endpoint (GET /api/health) — used by load balancers, Docker, and the frontend to verify the API is alive. |
| app/api/v1/catalog.py | Agent catalog endpoint — returns the list of available agents (UC1, UC2) with their capabilities and required inputs. |
| app/api/v1/workspaces.py | CRUD for workspaces — a workspace groups files and agent runs for one TPRA assessment engagement. |
| app/api/v1/files.py | File upload/download routes — handles ingestion of vendor security reports and retrieval of agent-generated outputs. |
| app/api/v1/agents.py | Agent execution routes — triggers UC1 or UC2 runs, returns run status and output file references. |
| app/api/v1/reviewer_log.py | Reviewer log routes — allows human reviewers to record approval/rejection decisions on individual findings between UC1 and UC2. |
| app/api/v1/audit.py | Audit trail routes — exposes the immutable event log of every action taken (who did what, when, on which finding). |

### app/services/ — Business Orchestration

| FILE | WHY IT EXISTS |
|---|---|
| app/services/agent_service.py | Central orchestrator: creates run records, loads inputs from storage, executes the appropriate agent workflow, persists outputs, and records audit events. Environment-agnostic — all infrastructure comes from the provider registry. |
| app/services/workspace_service.py | Workspace lifecycle management: creation, listing, deletion, and state transitions. |
| app/services/file_service.py | File handling logic: validates uploads, stores via the storage provider, and manages file metadata. |
| app/services/reviewer_log_service.py | Manages reviewer decisions: records approvals/rejections, validates state transitions, ensures only authorized reviewers can approve. |

| | |
|---|---|
| app/services/authz.py | Role-based access control enforcement: checks if the current user's role permits the requested action. |

### app/repositories/ — Data Access

| FILE | WHY IT EXISTS |
|---|---|
| app/repositories/base.py | Abstract base repository interface — defines the contract that all data stores (SQLite, Cosmos) must implement. |
| app/repositories/workspace_repository.py | Data access for workspace CRUD operations. |
| app/repositories/file_repository.py | Data access for file metadata (upload records, associations to workspaces). |
| app/repositories/run_repository.py | Data access for agent run records (status, timestamps, input/output references). |
| app/repositories/reviewer_log_repository.py | Data access for reviewer decision records. |
| app/repositories/audit_repository.py | Data access for the append-only audit event stream. |

### app/providers/ — Infrastructure Adapters

Each provider has a local/mock implementation (for Development Team dev) and an Azure implementation (for Client DaVinci). The registry selects which one based on `config.yaml`.

| FILE | WHY IT EXISTS |
|---|---|
| app/providers/base.py | Abstract interfaces for all five provider types (Storage, Metadata, DocIntelligence, LLM, Auth). |

| | |
|---|---|
| app/providers/registry.py | Factory/DI container: reads provider config, instantiates the correct implementation, and caches instances. This is why environment switching is a pure config change. |
| app/providers/storage/local.py | Stores files on the local filesystem (under .data/). Used in Development Team development. |
| app/providers/storage/azure_blob.py | Stores files in Azure Blob Storage. Used in Client DaVinci production. |
| app/providers/metadata/sqlite.py | SQLite-backed metadata store. Zero-config, file-based — perfect for local dev. |
| app/providers/metadata/cosmos.py | Azure Cosmos DB metadata store. Scalable, multi-region — for production. |
| app/providers/document_intelligence/local.py | Local document parser (basic text extraction from common formats). No cloud dependency. |
| app/providers/document_intelligence/azure.py | Azure Document Intelligence (Form Recognizer) for production-grade document parsing with layout understanding. |
| app/providers/llm/mock.py | Deterministic mock LLM that returns pre-defined responses. Enables offline development and predictable tests. |
| app/providers/llm/azure_openai.py | Azure OpenAI provider for GPT-based text generation in production. |
| app/providers/llm/foundry.py | Azure AI Foundry provider — invokes pre-deployed Foundry agents with managed prompts. |
| app/providers/auth/dev.py | Development auth: trusts identity from HTTP headers (no login required). For local testing only. |
| app/providers/auth/client_sso.py | Client SSO integration: validates JWT tokens from the client's OIDC identity provider. |

### app/agents/ — AI Agent Workflows

| FILE | WHY IT EXISTS |
|---|---|

| | |
|---|---|
| `app/agents/graph.py` | Minimal sequential state-graph workflow engine. Executes an ordered list of named step functions against shared state, records step traces (name, status, timing), and supports short-circuit via StopWorkflow exception. |
| `app/agents/agent_registry.py` | Loads agent definitions from `foundry/agents.yaml` — the single source of truth for agent names, models, and capabilities. |
| `app/agents/foundry_client.py` | Generic client for invoking Azure AI Foundry deployed agents via the Foundry SDK. |

**UC1 — Structured Findings Agent**

| FILE | WHY IT EXISTS |
|---|---|
| `app/agents/structured_findings/graph.py` | UC1 workflow definition: validate → authorize → load files → detect types → extract content → normalize findings → validate fields → capture reviewer log → build package (.xlsx) → exception report → reviewer log output → audit log → respond. |
| `app/agents/structured_findings/parsers.py` | Input file parsers: reads XLSX, CSV, JSON, and DOCX files into a common intermediate format for normalization. |
| `app/agents/structured_findings/normalizer.py` | Field normalization: assigns canonical IDs, standardizes field names, maps vendor-specific terminology to the TPRA schema. |
| `app/agents/structured_findings/validators.py` | Validation logic: checks required fields, flags exceptions for missing/invalid data, categorizes severity of issues. |
| `app/agents/structured_findings/package_builder.py` | Output builder: generates the structured findings Excel package, exception report, and reviewer log spreadsheet. |

**UC2 — Draft Report Agent**

| FILE | WHY IT EXISTS |
| --- | --- |
| app/agents/draft_report/graph.py | UC2 workflow definition: validate → authorize → load package → check approvals → load template → map sections → populate report (AI call) → detect missing inputs → missing input report → summary → audit log → respond. |
| app/agents/draft_report/report_builder.py | DOCX report generation: creates a formatted Word document from template + AI-generated narrative content. |
| app/agents/draft_report/template_mapper.py | Maps approved findings to specific sections of the TPRA report template based on finding category and severity. |

### tests/ — Test Suite

| FILE | WHY IT EXISTS |
| --- | --- |
| tests/conftest.py | Shared pytest fixtures: test client, mock providers, sample data factories. |
| tests/fixtures/sample_approved_findings.csv | Test input data: CSV file with sample findings for parser and normalizer tests. |
| tests/fixtures/sample_approved_findings.json | Test input data: JSON file with sample findings for integration tests. |
| tests/unit/test_parsers.py | Unit tests verifying each input parser (XLSX, CSV, JSON, DOCX) correctly extracts findings. |
| tests/unit/test_normalizer.py | Unit tests for field normalization: ID assignment, name standardization, deduplication. |
| tests/unit/test_validators.py | Unit tests for validation rules: required fields, exception flagging, edge cases. |
| tests/unit/test_agent_registry.py | Unit tests verifying agent definitions load correctly from YAML. |
| tests/integration/test_uc1_uc2_flow.py | Full integration test: exercises the complete UC1→UC2 pipeline end-to-end with mock providers. |

## Frontend React / TypeScript / Vite

| FILE | WHY IT EXISTS |
| --- | --- |
| `frontend/package.json` | NPM manifest: declares React 18, Vite 5, TypeScript, and all frontend dependencies. |
| `frontend/package-lock.json` | Lockfile: ensures every developer and CI gets identical dependency versions. |
| `frontend/tsconfig.json` | TypeScript compiler settings: strict mode, path aliases, JSX support. |
| `frontend/vite.config.ts` | Vite bundler config: dev server on port 5173, proxies `/api` requests to backend at localhost:8000. API base URL injectable via `VITE_API_BASE_URL` for different environments. |
| `frontend/index.html` | SPA entry point: the single HTML file that loads the React app bundle. |
| `frontend/Dockerfile` | Multi-stage build: Node.js builds the app, then nginx serves the static files with API proxying. |
| `frontend/nginx.conf` | Nginx configuration: serves the SPA (all routes → index.html) and reverse-proxies `/api` to the backend. |
| `frontend/.env.example` | Template for frontend environment variables (API base URL). |
| `frontend/src/main.tsx` | React entry point: mounts the root App component into the DOM. |
| `frontend/src/App.tsx` | Main application component: workspace management, file upload, agent run triggers, output viewer, and audit trail display. Includes a role selector for testing different user personas. |
| `frontend/src/config.ts` | Centralized config reader: reads Vite environment variables and exports typed configuration constants. |

| | |
|---|---|
| `frontend/src/api/client.ts` | HTTP client: wraps fetch with auth header injection (dev mode sends role headers; SSO mode sends JWT bearer token). |
| `frontend/src/styles.css` | Application styles: layout, component styling, responsive design rules. |
| `frontend/src/vite-env.d.ts` | TypeScript type declarations for Vite's special import features (env vars, asset imports). |

## Prompts

AI Prompts & Schemas

Versioned prompt repository — these define the AI agents' behavior. Stored in Git as source of truth and mirrored to Azure Blob for runtime access.

| FILE | WHY IT EXISTS |
|---|---|
| `prompts/shared/tpra_schema.json` | Canonical JSON Schema defining the Finding and Exception data models. Shared across both agents to ensure consistent output structure. |
| `prompts/structured_findings/v1/system.md` | UC1 system prompt: defines the agent's role, boundaries, rules of engagement, and expected behavior patterns. |
| `prompts/structured_findings/v1/user.md` | UC1 user prompt template: the dynamic prompt sent per invocation with input file details and workspace context. |
| `prompts/structured_findings/v1/output_schema.json` | UC1 output schema: forces the LLM to return structured JSON matching the expected findings package format. |
| `prompts/draft_report/v1/system.md` | UC2 system prompt: defines the report generation agent's role, writing style guidelines, and section requirements. |
| `prompts/draft_report/v1/user.md` | UC2 user prompt template: sent with the approved findings data for the AI to draft the TPRA narrative report. |

**Versioning:** Prompts are versioned (`v1/`) so new versions can be deployed side-by-side without breaking existing agent deployments.

## Foundry Azure AI Foundry

| FILE | WHY IT EXISTS |
|---|---|
| `foundry/agents.yaml` | Agent registry — the single source of truth defining all AI agents: their names, model deployments, prompt file references, and capabilities. |
| `foundry/structured-findings-agent.yaml` | Azure AI Foundry deployment definition for UC1: model ID, temperature, max tokens, tool configurations. |
| `foundry/draft-report-agent.yaml` | Azure AI Foundry deployment definition for UC2: model ID, temperature, max tokens, tool configurations. |

## Scripts Automation

| FILE | WHY IT EXISTS |
|---|---|
| `scripts/_deploy_common.sh` | Shared deployment routine: builds Docker image, pushes to ACR, publishes prompts to Blob, deploys to AKS. Sourced by stage-specific scripts. |
| `scripts/deploy_dev.sh` | Deploys to DEV environment (sets stage-specific vars, then calls _deploy_common.sh). |
| `scripts/deploy_qa.sh` | Deploys to QA environment. |
| `scripts/deploy_uat.sh` | Deploys to UAT environment. |

| | |
|---|---|
| `scripts/deploy_prod.sh` | Deploys to PROD environment. |
| `scripts/create_foundry_agents.py` | CI/CD script: reads agent definitions from `foundry/*.yaml`, creates or updates Azure AI Foundry deployed agents via the SDK, and prints the resulting agent IDs for configuration. |
| `scripts/upload_prompts.py` | Publishes the `prompts/` directory to Azure Blob Storage as an immutable mirror for runtime access and audit traceability. |

## Infrastructure Azure Bicep / IaC

| FILE | WHY IT EXISTS |
|---|---|
| `infra/bicep/main.bicep` | Infrastructure-as-Code template provisioning: Storage Account (TLS 1.2), Cosmos DB (session consistency), Document Intelligence, Key Vault (RBAC + soft-delete), Log Analytics workspace, and Application Insights. |
| `infra/bicep/dev.parameters.json` | Parameter values for the DEV stage deployment (resource names, SKUs, region). |
| `infra/bicep/qa.parameters.json` | Parameter values for the QA stage deployment. |
| `infra/bicep/uat.parameters.json` | Parameter values for the UAT stage deployment. |
| `infra/bicep/prod.parameters.json` | Parameter values for the PROD stage deployment. |

## CI/CD Pipeline Azure DevOps

## CI/CD Pipeline Azure DevOps

| FILE | WHY IT EXISTS |
| --- | --- |
| `azure-pipelines/azure-pipelines.yml` | Multi-stage CI/CD pipeline: runs tests → builds Docker images → deploys progressively through DEV → QA → UAT with manual approval gates. |

## Documentation Docs

| FILE | WHY IT EXISTS |
| --- | --- |
| `docs/ARCHITECTURE.md` | Detailed architecture description: clean architecture layers, agent registry pattern, provider abstraction, workflow engine design. |
| `docs/MIGRATION.md` | Step-by-step migration guide: how to switch from Development Team local development to Client DaVinci production (config changes, Azure setup, SSO integration). |
| `docs/repository-guide.html` | This file — comprehensive guide explaining the purpose of every file in the repository. |